Security Assignment

Carl Thulin
Linköping University
Linköping, Sweden
carth305@student.liu.se

TASK 0.1

HTTPS

HTTPS ensures secure communication between a client (browser) and a server (website) by using encryption, authentication, and integrity. It is based on HTTP but adds TLS for security.

TLS handshake

The client sends a "Client Hello" with supported encryption options. The server replies with a "Server Hello," its certificate, and a chosen cipher. After verifying the certificate, both generate a shared secret key and use it to encrypt all further communication.

X.509 certificates

An X.509 certificate is a digital document that verifies a server's or user's identity in TLS. It contains the subject's public key, issuer details, expiration date, and a signature from a trusted Certificate Authority (CA). Browsers verify this certificate to ensure secure connections.

TASK 0.2

Wireshark

Wireshark is a network protocol analyzer used to capture and inspect packets in real-time. It helps analyze network traffic, debug issues, and monitor security.

badssl.com

badssl.com is a testing website that provides various SSL/TLS configurations to help users and developers check how their browsers or applications handle different security scenarios.

crt.sh

crt.sh is a public certificate transparency (CT) log search tool that allows users to look up issued SSL/TLS certificates.

OpenSSL

OpenSSL is an open-source toolkit for TLS/SSL encryption and cryptographic operations. It allows users to generate keys, create and verify X.509 certificates, encrypt/decrypt data, and debug secure connections.

TASK 1

- **Expired**: Determine how the browser reacts when presented with a certificate that has expired ("notAfter" has a timestamp prior to today's date). This prevents security risks from outdated or potentially compromised certificates.
- **Wrong host**: The wrong host test checks if a browser detects a mismatch between the website's domain and the certificate's Common Name (CN) or Subject Alternative Name (SAN). This prevents man-in-the-middle attacks by ensuring the certificate is issued for the correct domain.
- **Self-signed**: The self-signed certificate test checks if a browser rejects certificates that are not issued by a trusted Certificate Authority (CA). This prevents attackers from creating untrusted certificates to impersonate legitimate websites.
- **Untrusted-root**: The untrusted root test checks if a browser rejects certificates issued by an unknown or untrusted Certificate Authority (CA). This ensures that only certificates from trusted CAs are accepted, preventing spoofing and man-in-the-middle attacks.
- **Revoked**: The revoked certificate test checks if a browser blocks certificates that have been revoked by the issuing Certificate Authority (CA). This prevents the use of compromised or invalid certificates, enhancing security against fraud and attacks.
- **Pinning test**: The pinning test checks if a browser properly enforces certificate pinning, ensuring it only accepts a specific certificate or CA for a website. This protects against man-in-the-middle attacks by preventing attackers from using fraudulent but valid certificates.
- **No-common-name**: The no common name test checks if a certificate is missing a Common Name (CN) or Subject Alternative Name (SAN), which are required for domain identification. This can cause browsers to reject the certificate, as it cannot be verified against the intended domain.
- **Client-cert-missing**: The client cert missing test checks if a server requires a client certificate for authentication but the client does not provide one. This ensures that access is restricted to authorized clients, enhancing security by enforcing mutual TLS authentication.
- **No-sct**: The no SCT test checks if a certificate is missing a Signed Certificate Timestamp (SCT), which is used to prove that the certificate has been logged in a public Certificate Transparency (CT) log. This helps prevent the use of fraudulent certificates by ensuring they are publicly auditable.
- **No-subject**: The no subject test checks if a certificate lacks a Subject field, which typically contains information about the entity the certificate is issued to (like the domain name). This can make the certificate invalid or untrustworthy, as it cannot properly identify the entity it is meant to secure.

TASK 2

I captured traces of all requested domains through Wireshark. For domains 2.6-2.12 I used a SSL key log file to be able to capture the certificates of these domains. Link to the traces: https://gitlab.liu.se/carth305/tdde53-security-assignment.git.

TASK 3

In the traces I can see the issuer on the row named "issuer:", the validality on "validity" and the size of the certificate on "Certificate Length".

#	Domain	Issuer	notBefore	notAfter	Size (bytes)
2.1	expired	COMODO RSA Domain Validation Secure Server CA	2015-04-09	2015-04-12	1359
2.2	wrong.host	R11	2025-01-07	2025-04-07	1271
2.3	self-signed	*.badssl.com	2025-02-11	2027-02-11	893
2.4	untrusted-root	BadSSL Untrusted Root Certificate Authority	2025-02-11	2027-02-11	1181
2.5	revoked	E6	2024-12-19	2025-03-19	904
2.6	liu.se	R11	2025-02-10	2025-05-11	1531
2.7	tsn.ca	Entrust Certification Authority - L1F	2024-06-06	2025-07-05	1920
2.8	seb.se	DigiCert Global G2 TLS RSA SHA256 2020 CA1	2024-10-22	2025-10-21	1835
2.9	scholar.google.com	Google Trust Services (WR2)	2025-02-03	2025-04-28	3599
2.10	facebook.com	DigiCert SHA2 High Assurance Server CA	2024-11-27	2025-02-25	1691
2.11	badssl.com	R11	2025-01-07	2025-04-07	1271
2.12	crt.sh	Sectigo RSA Organization Validation Secure Server CA	2025-01-20	2025-04-20	1861

TASK 4

I used the timestamps, CN and issuer fields to find the correct certifcate matching my trace. After I found the correct certificate, I looked at the validity and issuer field. I also needed to build a script in python that could count the number of expired and not expired certificates for me based on the domain name. Link to the script: https://gitlab.liu.se/carth305/tdde53-security-assignment.git

#	Domain	Issuer	notBefore (GMT)	notAfter (GMT)	# non-exp. certs	# exp. certs
2.1	expired	COMODO RSA Domain Validation Secure Server CA	Apr 9 00:00:00 2015	Apr 12 23:59:59 2015	12	12
2.2	wrong.host	Let´s Encrypt (R11)	Jan 7 03:02:51 2025	Apr 7 03:02:50 2025	12	12
2.3	self-signed	NA	NA	NA	NA	NA
2.4	untrusted-root	NA	NA	NA	NA	NA
2.5	revoked	Let´s Encrypt (E6)	Dec 19 18:21:15 2024	Mar 19 18:21:14 2025	12	12
2.6	liu.se	Let´s Encrypt (R11)	Feb 10 05:38:42 2025	May 11 05:38:41 2025	4324	3781
2.7	tsn.ca	Entrust Certification Authority - L1F	Jun 6 18:44:44 2024	Jul 5 18:44:42 2025	251	24
2.8	seb.se	DigiCert Global G2 TLS RSA SHA256 2020 CA1	Oct 22 00:00:00 2024	Oct 21 23:59:59 2025	1248	115
2.9	scholar.google.com	Google Trust Services (WR2)	Feb 3 08:36:05 2025	Apr 28 08:36:04 2025	1152	1351
2.10	facebook.com	DigiCert SHA2 High Assurance Server CA	Nov 27 00:00:00 2024	Feb 25 23:59:59 2025	2093	2037
2.11	badssl.com	Let´s Encrypt (R11)	Jan 7 03:02:51 2025	Apr 7 03:02:50 2025	12	12
2.12	crt.sh	Sectigo RSA Organization Validation Secure Server CA	Jan 20 00:00:00 2025	Apr 20 23:59:59 2025	104	8

TASK 5

I downloaded the certificates in PEM format from crt.sh and looked how big that file was.

Case	Domain	Size of PEM form. cert (bytes)
2.6	liu.se	2130
2.7	tsn.ca	2654
2.8	seb.se	2045
2.9	scholar.google.com	4929
2.10	facebook.com	2126
2.11	badssl.com	1777
2.12	crt.sh	2577

TASK 6

I got the human-understandable output from OpenSSL in bytes through the command "openssl x509 -in [filename] -text -noout | wc -c". The human-readable output from OpenSSL is larger than the PEM certificate format because it includes descriptive labels and more detailed information for human inspection. For example, a PEM certificate is smaller due to the added text such as “Issuer:”, “Subject:”, and other labels in the ouput from OpenSSL. The raw certificate format is compact and machine-readable, while the OpenSSL output is designed to be more informative for humans, thus increasing its size.

Case	Domain	human understandable output from OpenSSL (bytes)
2.6	liu.se	4997
2.7	tsn.ca	3924
2.8	seb.se	4211
2.9	scholar.google.com	6132
2.10	facebook.com	4259
2.11	badssl.com	3868
2.12	crt.sh	5404

TASK 7

- **Version**: 3 (0x2)
  The version of the X.509 certificate (version 3).
- **Serial Number**:
  03:48:41:b7:d7:f4:23:ca:7a:f8:a5:41:47:05:67:2d:e1:58
  A unique identifier assigned to the certificate by the issuing Certificate Authority (CA).
- **Signature Algorithm**: sha256WithRSAEncryption
  The algorithm used for signing the certificate, in this case, SHA-256 with RSA encryption.
- **Issuer**:
  The entity that issued the certificate, including:
  - **Common Name**: R11
  - **Organization Name**: Let’s Encrypt
  - **Country Name**: US
- **Validity**:
  The time period for which the certificate is valid:
  - Not Before: Feb 10 05:38:42 2025 GMT
  - Not After: May 11 05:38:41 2025 GMT
- **Subject**:
  The entity to which the certificate was issued:
  - **Common Name**: liu.se
- **Subject Public Key Info**:
  Information about the public key:
  - **Public Key Algorithm**: RSA
  - **RSA Public Key**: 4096 bit key length
  - **Exponent**: 65537 (0x10001)
- **X509v3 Extensions**:
  Extra attributes of the certificate:
  - **Key Usage**:
    Digital Signature, Key Encipherment
  - **Extended Key Usage**:
    TLS Web Server Authentication
  - **Basic Constraints**:
    * CA: FALSE
    * Path Length Constraint: None
  - **Subject Key Identifier**: 7E:6C:70:9F:90:F2:90:9A:36:43:AC:88:63:A5:32
  - **Authority Key Identifier**:
    * Key Identifier: 66:35:9E:DB:67:31:0C:0E:72:62:3F:9E:90:AB:06:44:
    * Authority Cert Issuer: `CN=R11, O=Let’s Encrypt, C=US`
  - **Authority Information Access**:
    * OCSP: `http://ocsp.int-x3.letsencrypt.org`
    * CA Issuers: `http://cert.int-x3.letsencrypt.org`
  - **Subject Alternative Name**:
    * DNS: liu.se
    * DNS: www.liu.se
  - **Certificate Policies**:
    * Policy Identifier: 1.3.6.1.5.5.7.3.1

REFLECTION

The choice of CA and the validity period used in certificates are crucial for the overall security of a system. A well-known and trusted CA provides confidence that the certificate can be trusted. However, choosing a lesser known or untrusted CA could lead to vulnerabilities if an attacker can compromise that CA.

As for the validity period, shorter durations could increase security by ensuring certificates are updated more frequently. This reduces the risk of using outdated cryptographic algorithms or compromised certificates. It also helps to ensure that the certificate reflects the most up-to-date information about the domain it represents. The difference in validity periods observed across certificates shows how organizations approach security. Some may opt for longer periods to reduce administrative overhead, while others may choose shorter periods to prioritize security.

In conclusion, both the choice of CA and the length of the certificate validity period play an important role in ensuring security. Shorter validity periods are preferable for reducing risks, and trusted CAs are vital to establish the authenticity and integrity of the certificate chain.

Case	Domain	Crt.sh ID (if any)	Issuing CA	Validity period	# non-exp. certs	# exp. certs	Size of cert from trace (bytes)	Size of PEM form. cert (bytes)
2.1	expired.badssl.com	7176112	COMODO RSA Domain Validation Secure Server CA	Apr 9 00:00:00 2015 GMT - Apr 12 23:59:59 2015 GMT	12	12	1359	-
2.2	wrong.host.badssl.com	16414864904	Let´s Encrypt (R11)	Jan 7 03:02:51 2025 GMT - Apr 7 03:02:50 2025 GMT	12	12	1271	-
2.3	self-signed.badssl.com	NA	*.badssl.com	2025-02-11 - 2027-02-11	NA	NA	893	-
2.4	untrusted-root.badssl.com	NA	BadSSL Untrusted Root Certificate Authority	2025-02-11 - 2027-02-11	NA	NA	1181	-
2.5	revoked.badssl.com	16168657929	Let´s Encrypt (E6)	Dec 19 18:21:15 2024 GMT - Mar 19 18:21:14 2025 GMT	12	12	904	-
2.6	liu.se	16648379256	Let´s Encrypt (R11)	Feb 10 05:38:42 2025 GMT - May 11 05:38:41 2025 GMT	4324	3781	1531	2130
2.7	tsn.ca	14509477562	Entrust Certification Authority - L1F	Jun 6 18:44:44 2024 GMT - Jul 5 18:44:42 2025 GMT	251	24	1920	2654
2.8	seb.se	15040402240	DigiCert Global G2 TLS RSA SHA256 2020 CA1	Oct 22 00:00:00 2024 GMT - Oct 21 23:59:59 2025 GMT	1248	115	1835	2045
2.9	scholar.google.com	16845103669	Google Trust Services (WR2)	Feb 3 08:36:05 2025 GMT - Apr 28 08:36:04 2025 GMT	1152	1351	3599	4929
2.10	facebook.com	15512577850	DigiCert SHA2 High Assurance Server CA	Nov 27 00:00:00 2024 GMT - Feb 25 23:59:59 2025 GMT	2093	2037	1691	2126
2.11	badssl.com	16414864904	Let´s Encrypt (R11)	Jan 7 03:02:51 2025 GMT - Apr 7 03:02:50 2025 GMT	12	12	1271	1777
2.12	crt.sh	16338712982	Sectigo RSA Organization Validation Secure Server CA	Jan 20 00:00:00 2025 GMT - Apr 20 23:59:59 2025 GMT	104	8	1861	2577